MODELS OF COMPUTATION (BACSE201)

PROJECT REPORT

AUTOSEC

Counterexample-Driven Automata and Syntax Demo

Team Members

Bhoumik Singh Rajput | 25BYB0120

Vihaan Jinesh Jain | 25BYB0130

Faculty

Dr. Dharmendra Singh Rajput

School of Computer Science Engineering and Information Systems

Vellore Institute of Technology, Vellore

Academic Year 2026–2027

24 October 2026

Abstract

Pattern occurrence and syntactic membership describe different properties of an input. Conflating them can obscure why a command-like string is accepted or rejected. AUTOSEC is a browser-local demonstrator that separates these observations and explains differences between two regular rules using a shortest counterexample. A TypeScript engine parses supported regular expressions, constructs Thompson ε-nondeterministic finite automata, computes ε-closures, determinises the machines, and minimises their deterministic forms. A second pipeline tokenises a restricted nested command language and performs predictive parsing using calculated FIRST/FOLLOW sets and an LL(1) table. Product-automaton breadth-first search returns either language equivalence or a shortest string accepted by exactly one rule. The interface exposes only two tasks, Check text and Compare rules; explanatory traces remain optional. Verification on 24 October 2026 recorded 90 passing automated tests and a successful production build. Eight independently specified languages were checked on all 127 binary strings of length at most six. Actual application captures document valid syntax, a missing flag value, and the witness `--a`. The contribution is a focused integration of established algorithms and contrasting language properties, rather than a new automata algorithm, a full shell, or a threat detector. No learning-effectiveness improvement is claimed without a user study.

Index Terms: formal languages, finite automata, regular expressions, predictive parsing, context-free grammar, shortest counterexample, educational software.

Report organisation

I. Introduction and Related Work	3
II. Formal Definitions and Recognition Semantics	4
III. Methodology and System Architecture	5
IV. Regular-Language Construction	6
V. Lexical Analysis and Predictive Parsing	7
VI. Shortest-Counterexample Method	8
VII. Actual Application Screenshots	9
VIII. Evaluation and Observed Results	10
IX. Discussion, Limitations and Conclusion	11
References	12

Project scope

The implementation demonstrates regular-language recognition, nested syntax and a constructive difference between two languages. All inputs are treated as text. The small interface is intentional: definitions, diagrams and methodological detail are supplied in this report without adding dashboard controls to the application. A recent textbook provides the course context for automata and parsing [1].

I. Introduction and Related Work

A. Problem statement

The strings `inspect --tag sample` and `inspect --tag` both contain a flag-shaped substring. Only the first provides the value required by the project grammar. Similarly, balanced command groups require an account of nested structure rather than a single local flag match. AUTOSEC therefore reports pattern occurrence and grammatical acceptance independently, then offers a second experiment: identify the smallest observable disagreement after changing a regular rule.

Recent educational work explains regex-to-automaton transformations [2], nondeterministic rejection [3], and machine/stack behaviour [4]. Recipe-based diagnostics support regex development [5]. These publications motivate explicit explanations, but their reported findings are not evidence that AUTOSEC improves learning. Table I compares documented emphases rather than asserting that a feature is absent from every version of another tool.

TABLE I. DOCUMENTED EMPHASIS AND AUTOSEC DESIGN POSITION

Work	Documented emphasis	AUTOSEC emphasis
FSM visualisation [2]-[4]	Transformations, computation graphs and machine/stack execution.	A command with two verdicts, then a shortest rule disagreement.
Recipe support [5]	Regex design process and targeted diagnostic feedback.	A small set of prepared contrasts and optional computed steps.
automata [6]	Python API, simulation, conversion, language operations and equivalence.	Direct browser forms without coding an automata API.
AALpy [7]	Active model learning, testing and equivalence oracles.	Exact product search between two supplied regex-derived DFAs.

B. Distinction and objectives

The distinguishing design is the combination of **two independent verdicts on the same command** and **a shortest witness between two explicit rules**, presented in a two-task interface. The automata library already implements major transformations and language operations [6]; AALpy already uses counterexamples in model learning [7]. Consequently, neither automata conversion nor counterexample generation is claimed as a new invention. AUTOSEC connects these ideas to one fixed nested command example with minimal setup.

The objectives are to compute genuine results, preserve source offsets, explain the first syntax failure, and make a rule disagreement reproducible. The comparison is qualitative and source-based. No benchmark establishes superior accuracy, usability or speed over the cited systems.

II. Formal Definitions and Recognition Semantics

A. Symbols, languages and finite-state machines

An **alphabet** Σ is a finite set of symbols. A **string** w is a finite sequence over Σ; ε denotes the empty string and |w| its length. A **language** is a set of strings, L ⊆ Σ*. A regular expression denotes a regular language through literals, concatenation, union and repetition. In the supported syntax, `r+` denotes one or more repetitions and `r?` denotes zero or one. These operators do not execute input.

A **DFA** is M = (Q, Σ, δ, q_0, F), where Q is a finite state set, δ is a deterministic transition function, q_0 is the initial state, and F is the accepting set. Its extended transition δ* consumes a string. An **ε-NFA** permits several possible successor states and ε-transitions that consume no character. Its active configuration is a set of states. **ε-closure(S)** includes S and all states reachable from S by zero or more ε-transitions.

$$L(M) = \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\} \quad (1)$$

$$\varepsilon\text{-closure}(S) = \{q \mid \exists s \in S,\ s \rightarrow_{\varepsilon}^* q\} \quad (2)$$

B. Syntax and distinguishing inputs

A **CFG** is G = (V, T, P, S): nonterminals V, terminals T, productions P, and start symbol S. A **token** records a lexical category, original text and interval [start, end). An **LL(1) parser** chooses a production using its stack top and one lookahead token. A **PDA** augments finite control with a stack; this explains the memory required by arbitrarily nested groups. The implemented predictive parser is the demonstrated stack-based recogniser, not a general user-editable PDA simulator.

TABLE II. NOTATION USED THROUGHOUT THE REPORT

Symbol	Meaning
Σ*	All finite strings over Σ, including ε.
L(r), L(G)	Languages denoted by a regex and generated by the grammar.
δ*(q, w)	State reached after consuming w from state q.
A △ B	Symmetric difference: strings in exactly one language.
[s, e)	Start included; end excluded. Source offsets are zero-based.
$	End-of-input marker, not a character in the command.

Pattern search asks whether a substring belongs to L(r). Syntax asks whether the complete token sequence belongs to L(G). Rule comparison uses whole-string membership in both regex languages. A **distinguishing string** lies in their symmetric difference. A shortest witness minimises length; it need not be unique. Raw matching may find `--tag` inside `inspect "--tag"`, while the lexer correctly treats the quoted text as one value.

III. Methodology and System Architecture

A. Design and verification procedure

The project follows a construction-and-verification methodology: define a restricted language; implement explicit machine representations; integrate independent analysis paths; verify transformations against independent predicates; and inspect the running interface. The work is a software artefact evaluation, not an experiment measuring student learning. Programming-oriented automata activities provide related educational context [8], but no participant outcomes are attributed to this project.

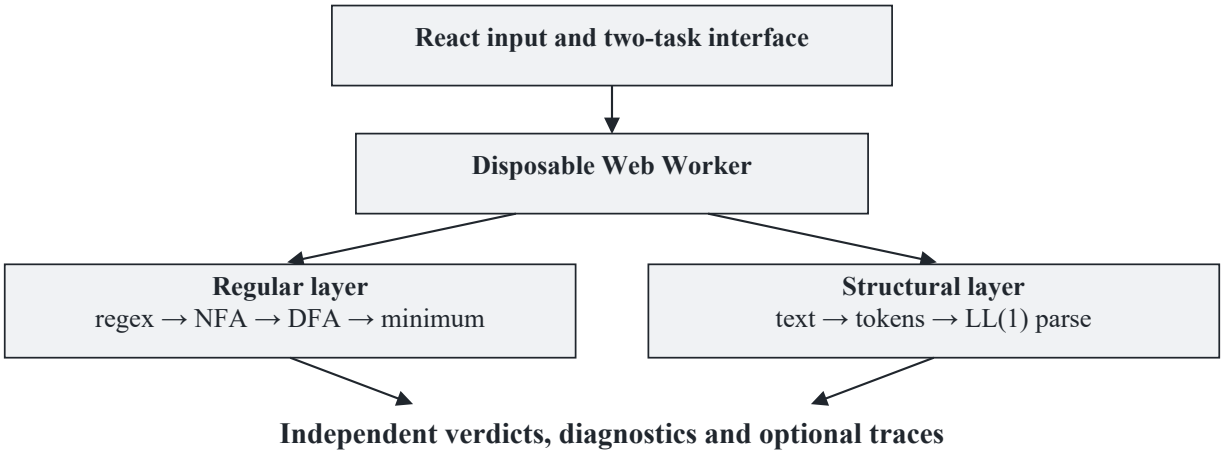


Fig. 1. Independent regular and structural pipelines inside a local worker.

B. Computation protocol

React holds input and result state. Submitting a form creates a disposable module Web Worker and sends `{id, task, data}`. The dispatcher calls `analyse()` or compiles both comparison rules before `compareDfa()`. The worker returns serialisable machine, token, trace and diagnostic data. The main thread renders verdicts; it does not manufacture an answer from the selected example name. Editing clears the previous result, and switching views disposes of pending computation.

TABLE III. METHODOLOGY AND CORRESPONDING ARTEFACT

Stage	Concrete procedure	Evidence
Specification	Fix regex subset, token categories and nested grammar.	Definitions and productions.
Construction	Generate automata and predictive decisions from inputs.	Machine edges, sets and traces.
Verification	Use independent predicates and exact comparisons.	90 tests and worker fixtures.
Presentation	Exercise examples and capture real rendered results.	Cursor-free browser captures.

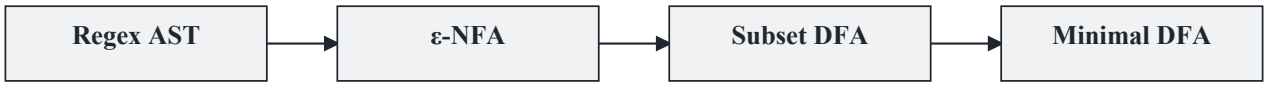
C. Resource and operational boundaries

The engine admits 98 ASCII symbols, at most 128 regex characters, 2,048 input characters, 256 NFA states and 512 DFA states. A request exceeding 12 seconds is terminated. Cancellation or resource exhaustion produces **analysis incomplete**, not rejection or equivalence. Ordinary use of the installed local build requires no external service. Inputs are not sent to a backend, executed as commands, or used to visit entered URLs. Unsupported regex constructs are diagnosed rather than approximated silently.

IV. Regular-Language Construction

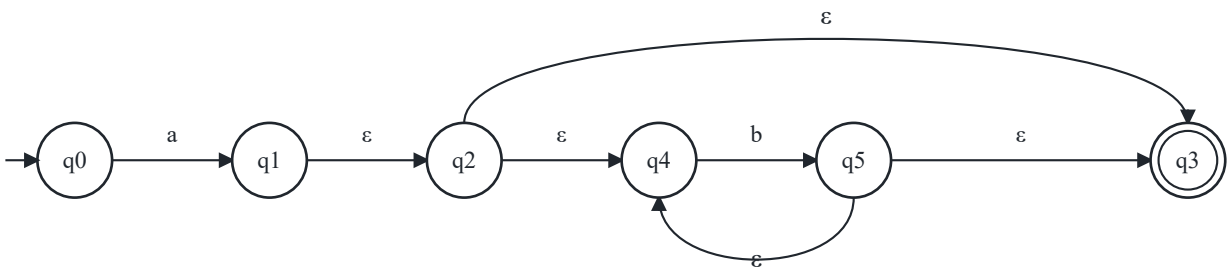
A. Parsing and Thompson construction

`parseRegex()` builds an abstract syntax tree with alternation, concatenation and repetition precedence. `thompson()` recursively constructs entry/exit fragments. Literal fragments add labelled edges; concatenation connects fragments by ε; alternation introduces ε-branches; repetition adds the required skip or loop. Character classes such as `[a-z]` expand into supported symbols. Backreferences, lookarounds, anchors and counted repetitions are rejected by this subset.



B. Worked construction for ab*

The expression `ab*` denotes an `a` followed by zero or more `b` symbols. Figure 2 is drawn from stored engine edges, not a hand-selected substitute machine. After reading `a`, ε-closure reaches {1,2,3,4}; this set contains accepting NFA state 3. Subset construction records each reachable closed set as one DFA state, including the empty rejecting subset when reachable.



All stored transitions shown; double circle marks acceptance.

Fig. 2. Actual Thompson ε-NFA for ab*. State IDs and edges follow the worker fixture.

TABLE IV. REACHABLE DFA SUBSETS FOR ab*

DFA state	Represented NFA set	Accepting?
q0	{0}	No
q1	{1,2,3,4}	Yes
q2	{}	No
q3	{3,4,5}	Yes

C. Minimisation and implementation limits

Partition refinement separates accepting from rejecting states, repeatedly splitting blocks with distinguishable transitions. For this fixture, DFA states 1 and 3 merge: both accept and have the same future language. The measured sequence is **6 NFA → 4 DFA → 3 minimal-DFA states**. The implementation uses a Hopcroft-style worklist with straightforward set scans; it does not claim the optimised textbook running-time bound. Extended derivative-based regex matching supports constructs intentionally outside this demo [9]. Such research is a scope comparison, not a component dependency or a reason to claim full regex compatibility.

V. Lexical Analysis and Predictive Parsing

A. Fixed nested command language

The lexer recognises WORD, lowercase FLAG, double-quoted QUOTED values, `;`, `(` and `)`. Quoted values permit escaped quotes and backslashes. Adjacent words, flags and quoted values require whitespace. A flag always requires a value. Tokens preserve source offsets so that a failure can be related to the original text. Lexing and parsing remain separate: if lexing fails, the parser does not accept a partial stream.

```
Script    → Statement Rest
Rest      → ; Statement Rest | ε
Statement → WORD Args | ( Script )
Args      → Arg Args | ε
Arg       → FLAG Value | Value
Value     → WORD | QUOTED
```

B. Calculated decisions

FIRST(A) contains terminals that can begin strings derived from A, with ε when A is nullable. FOLLOW(A) contains terminals that can immediately follow A; `$` marks end-of-input. `grammarInfo()` calculates these sets to a fixed point, fills the predictive table and detects duplicate assignments. The current grammar yields no table conflicts. The empty production is stored as an empty array, rather than an input character.

TABLE V. FIRST/FOLLOW SETS COMPUTED FROM THE FIXED GRAMMAR

Nonterminal	FIRST	FOLLOW
Script	{(, WORD}	{$,)}
Rest	{;, ε}	{$,)}
Statement	{(, WORD}	{$,), ;}
Args	{FLAG, QUOTED, WORD, ε}	{$,), ;}
Arg	{FLAG, QUOTED, WORD}	{$,), ;, FLAG, QUOTED, WORD}
Value	{QUOTED, WORD}	{$,), ;, FLAG, QUOTED, WORD}

C. Dry run and stack interpretation

For `inspect --tag sample; (scan file)`, the worker records **8 tokens and 25 parser actions**. The stack begins with Script over `$`. Expansions push right-hand sides in reverse order, making the leftmost symbol the next one processed. An opening group leaves a closing `)` pending until its inner Script finishes.

TABLE VI. FIRST FOUR ACTIONS FOR THE VALID COMMAND

Step	Stack top first	Applied action
1	Script $	Script → Statement Rest
2	Statement Rest $	Statement → WORD Args
3	WORD Args Rest $	Match WORD: "inspect"
4	Args Rest $	Args → Arg Args

For `inspect --tag`, Value remains pending at end-of-input (offset 13); the pattern still matches. The missing parenthesis fails at offset 32. Engine diagnostics retain these zero-based offsets. POSIX lexing [10] and verified lexer work [11] address stronger proof obligations than this tested implementation; no mechanised verification is claimed.

VI. Shortest-Counterexample Method

A. Product construction

Let A and B be generated DFAs. Each product node is a pair (p, q). A symbol advances both components; an undefined transition enters an implicit rejecting sink. A node distinguishes the languages exactly when one component accepts and the other rejects. The search uses the union of both alphabets and visits each reachable pair once.

$$L(A) \triangle L(B) = (L(A) - L(B)) \cup (L(B) - L(A)) \quad (3)$$

$$|w^*| = \min \{|w| : w \in L(A) \triangle L(B)\} \quad (4)$$

```
queue ← [(startA, startB)]
for each pair in queue, in discovery order:
    if acceptingA(pair.left) XOR acceptingB(pair.right):
        return the input reconstructed through parent links
    for each symbol in the shared alphabet:
        enqueue the successor pair if it has not been seen
return equivalent
```

B. Shortestness argument and observed witness

Every product edge consumes one character, so breadth-first traversal processes nondecreasing input lengths. The first acceptance-disagreeing node therefore has a shortest path from the initial pair. Keeping one shortest predecessor per pair is sufficient because future acceptance depends on the pair, not its earlier path. If the complete reachable product is exhausted without disagreement, the symmetric difference is empty. Interruption is reported as incomplete.



After --a: A accepts; B rejects. Only the witness path is drawn.

Fig. 3. Paired DFA run along --a. Numeric labels belong to separate machines.

TABLE VII. COMPUTED ACCEPTANCE ALONG THE SHORTEST WITNESS

Prefix	A state / acceptance	B state / acceptance
ε	q0 / reject	q0 / reject
-	q1 / reject	q1 / reject
--	q3 / reject	q3 / reject
--a	q4 / accept	q4 / reject

The default A = `--[a-z]+` permits one letter; B = `--[a-z][a-z]+` requires two. The observed witness is `--a`, found after **5 processed pairs**. Shorter strings satisfy neither rule. Equal numeric labels in separate machines do not imply equal acceptance. The product search is O(kmn) time and O(mn) storage for m and n completed DFA states and alphabet size k; this bound excludes regex compilation. AALpy's learning/oracle workflow [7] is a different task from this exhaustive comparison of two explicit finite machines.

VII. Actual Application Screenshots

These genuine application captures were taken on 24 October 2026 at `http://127.0.0.1:4173/`, using an 1100 × 900 desktop viewport. No mockup, cursor overlay or altered result was used. Full originals accompany the repository; Figure 5 is a directly captured viewport excerpt. Images are resized only for placement.

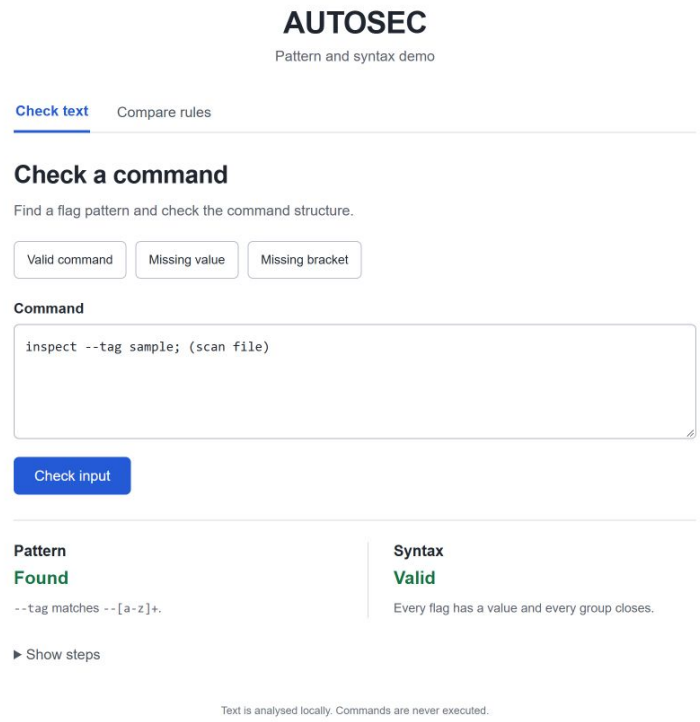


Fig. 4. Actual Check text view: the flag is found and the nested command is valid.

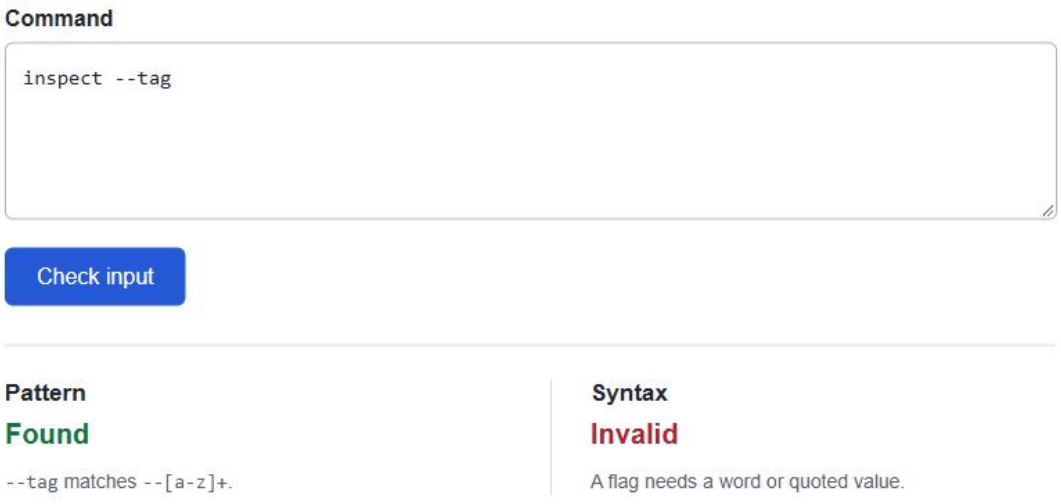


Fig. 5. Actual viewport excerpt: the flag remains present while a missing value makes syntax invalid.

Both results are computed. A pattern match proves neither complete syntax nor malicious intent. The contrast requires no graph editing or saved-case workflow.

VIII. Evaluation and Observed Results

A. Verification protocol

The suite separates engine tests from DOM interaction tests. Eight independent language predicates are evaluated on every string over {a,b} of length 0 through 6: 1 + 2 + 4 + 8 + 16 + 32 + 64 = 127 strings per language. Each expected answer is compared against NFA, DFA and minimal-DFA simulation, producing **3,048 recognition checks** within those parameterised test cases. Exact product checks additionally compare language preservation; finite enumeration alone is not treated as a proof for arbitrary strings.



8 languages × 127 strings × 3 recognisers = 3,048 recognition checks

Fig. 6. Passing automated test cases and the embedded recognition-check count.

TABLE VIII. OBSERVED VERIFICATION RESULTS, 24 OCTOBER 2026

Check	Observed result	Interpretation
Engine / DOM suite	83 / 7 passing	90 test cases; not 90 user participants.
Production build	Passed	TypeScript and Vite build successful.
ab* construction	6 → 4 → 3 states	NFA, DFA and minimal DFA.
Default comparison	--a; 5 processed pairs	A accepts and B rejects.
a+ versus aa*	Equivalent	Reachable product exhausted.
a* versus a+	Empty witness ε	Initial acceptance disagrees.

B. Behavioural cases and evidence provenance

TABLE IX. TWO INDEPENDENT OUTCOMES ON THE SAME INPUT

Input	Flag found?	Syntax accepted?
inspect --tag sample; (scan file)	Yes	Yes
inspect --tag	Yes	No
inspect file	No	Yes
(inspect file	No	No
inspect "--tag"	Yes	Yes

The 24 October browser session reproduced valid and missing-value results, the default witness and trace, and missing-bracket behaviour at 390px. No console warnings/errors were observed; the mobile document width equalled its 390px viewport. The 23 October real-browser review separately verified cancellation, equivalence, an empty witness and invalid-regex diagnostics. DOM cancellation tests also passed in the current run. Node worker fixtures verify the built asset and serialisation; they are not browser timing benchmarks. No threat-detection accuracy, learning-gain percentage or performance superiority was measured.

IX. Discussion, Limitations and Conclusion

A. Interpretation and contribution

The counterexample result converts a rule difference into a concrete, minimal observation. The paired syntax result addresses a different misconception: a locally matching substring does not validate an entire nested input. AUTOSEC's distinction is therefore the selected combination and presentation of these experiments. It does not supersede the broader APIs of automata [6], the active-learning facilities of AALpy [7], or the transformation and execution visualisations in FSM [2]-[4].

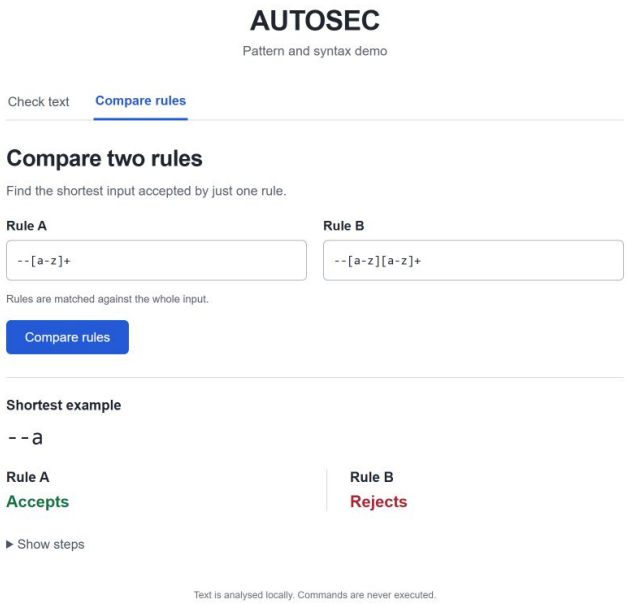


Fig. 7. Actual Compare rules view: --a is accepted only by Rule A.

B. Threats to validity and future evaluation

The prepared examples are selected demonstrations, not a random sample of user behaviour. ASCII and state bounds restrict the domain; token and regex semantics are intentionally narrower than production shells and regex dialects. Tests share some implementation infrastructure, although independent predicates and exact product checks reduce reliance on a single result path. Browser observations sample one browser and two viewport sizes. Labels, keyboard interaction and simple disclosure controls support inspection, but WCAG 2.2 [12] compliance has not been comprehensively audited.

A future controlled study could compare unaided and tool-assisted explanations of pattern/syntax disagreement, then measure witness reasoning, task time and error rates. Educational activity design [8] and concise diagnostics [5] are relevant starting points, not substitutes for participant evidence. Adding features is unnecessary for the present two-minute demonstration.

C. Conclusion

AUTOSEC implements real automata construction, predictive parsing and exhaustive product search behind a small interface. The observed tests, worked traces and clean captures make the selected results reproducible. The report provides the definitions and methodological detail; the application keeps the demonstration focused. Claims remain limited to the implemented language subset and verified observations.

References

[1] P. Linz and S. H. Rodger, *An Introduction to Formal Languages and Automata*, 7th ed. Burlington, MA, USA: Jones & Bartlett Learning, 2023. [Online]

[2] M. T. Morazán and T. Minić, "Finite-State Automaton To/From Regular Expression Visualization," *Electronic Proceedings in Theoretical Computer Science*, vol. 405, pp. 36-55, 2024. doi: 10.4204/EPTCS.405.3. [Online]

[3] O. Kempinski and M. T. Morazán, "Visualizing Why Nondeterministic Finite-State Automata Reject," *arXiv preprint arXiv:2310.08025*, presented at the Scheme and Functional Programming Workshop, 2023. doi: 10.48550/arXiv.2310.08025. [Online]

[4] M. T. Morazán, D. A. K. Fields, A. M. Garced, and T. Minić, "Visual Execution and Validation of Finite-State Machines and Pushdown Automata," *Electronic Proceedings in Theoretical Computer Science*, vol. 424, pp. 87-108, 2025. doi: 10.4204/EPTCS.424.5. [Online]

[5] M. T. Morazán et al., "A Design Recipe and Recipe-Based Errors for Regular Expressions," *Electronic Proceedings in Theoretical Computer Science*, vol. 424, pp. 25-48, 2025. doi: 10.4204/EPTCS.424.2. [Online]

[6] C. Evans and E. W. Robson, "automata: A Python package for simulating and manipulating automata," *Journal of Open Source Software*, vol. 8, no. 90, Art. no. 5759, 2023. doi: 10.21105/joss.05759. [Online]

[7] E. Muškardin, B. K. Aichernig, I. Pill, A. Pferscher, and M. Tappler, "AALpy: An active automata learning library," *Innovations in Systems and Software Engineering*, vol. 18, no. 3, pp. 417-426, 2022. doi: 10.1007/s11334-022-00449-3. [Online]

[8] J. Dochkova-Todorova and I. Donchev, "STEM Approach to Build Competences from Automata and Languages Theory," *Science Series Innovative STEM Education*, vol. 6, pp. 35-54, 2024. doi: 10.55630/STEM.2024.0604. [Online]

[9] I. E. Varatalu, M. Veanes, and J.-P. Ernits, "Derivative Based Extended Regular Expression Matching Supporting Intersection, Complement and Lookarounds," *arXiv preprint arXiv:2309.14401*, 2023. doi: 10.48550/arXiv.2309.14401. [Online]

[10] C. Tan and C. Urban, "POSIX Lexing with Bitcoded Derivatives," *Proc. 14th Int. Conf. Interactive Theorem Proving (LIPIcs)*, vol. 268, pp. 27:1-27:18, 2023. doi: 10.4230/LIPIcs.ITP.2023.27. [Online]

[11] D. Egolf, S. Lasser, and K. Fisher, "Verbatim++: Verified, Optimized, and Semantically Rich Lexing with Derivatives," *Proc. 11th ACM SIGPLAN Int. Conf. Certified Programs and Proofs*, pp. 27-39, 2022. doi: 10.1145/3497775.3503694. [Online]

[12] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG) 2.2," *W3C Recommendation*, Oct. 5, 2023. [Online] Accessed: Oct. 24, 2026.